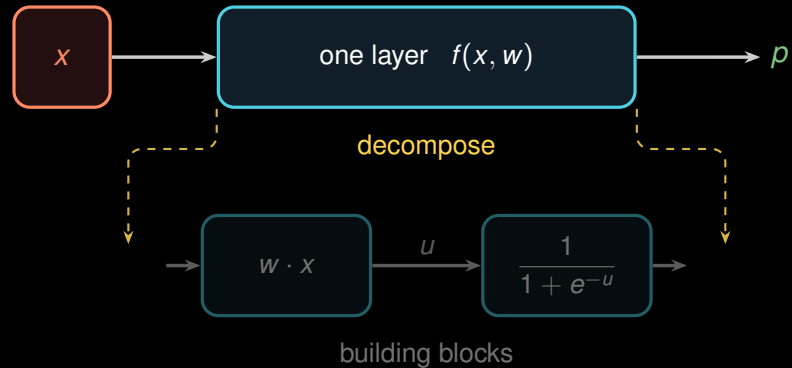


Multilayer networks

DECOMPOSITION OF A SINGLE LINEAR LAYER

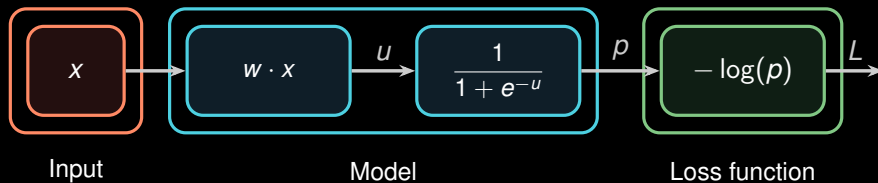
A linear layer: weigh the input $x \in \mathbb{R}^d$, then map the score to a probability $p = \sigma(w \cdot x)$.



LINEAR CLASSIFIER

A linear classifier can be broken down into:

- ▶ an input,
- ▶ a function of the input,
- ▶ a loss function.

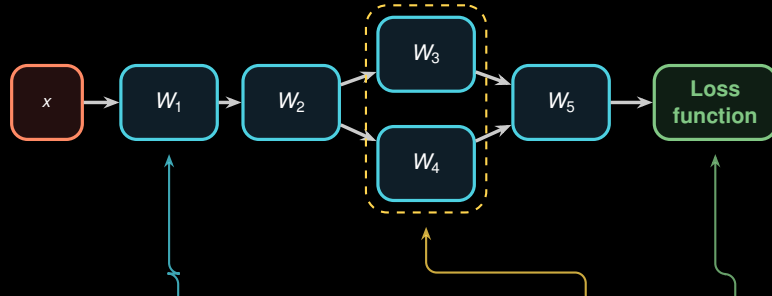


That middle group is one layer. Stack several of them and the model becomes a multilayer network.

COMPLEX FUNCTIONS AND COMPUTATION GRAPHS

Functions can be made arbitrarily complex, subject only to memory and compute:

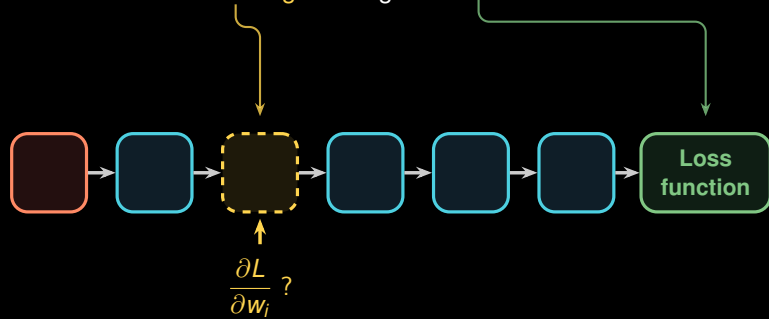
$$h = \sigma(W_2 \sigma(W_1 x))$$
$$f(x, W) = \sigma(W_5 [\sigma(W_3 h) ; \sigma(W_4 h)])$$



We can use any differentiable function as a layer, the graph may branch, and the loss comes last.

COMPUTING GRADIENTS IN A COMPLEX FUNCTION

- ▶ We are learning complex models.
 - With millions or billions of **parameters**.
 - The **loss** sits at the very end. The **weights** sit everywhere inside.
- ▶ So: how does an **internal weight** change the **loss** at the end?

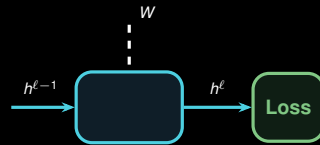


Backpropagation

FORWARD PASS AND BACKWARD PASS

Given a computation graph, the training algorithm performs two passes over it:

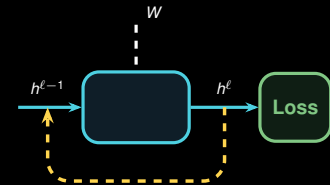
Forward pass



The **forward pass** calculates the model's outputs.

- ▶ at layer ℓ , take $h^{\ell-1}$ and produce h^ℓ using W ;
- ▶ **retain every h^ℓ** , which the backward pass will need;
- ▶ repeat layer by layer until the **loss**.

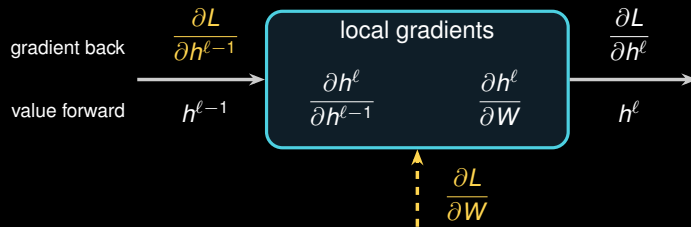
Backward pass



The **backward pass**, or **backpropagation**, calculates the gradients for each module.

- ▶ it is **recursive**;
- ▶ it progresses back through the modules;
- ▶ and it answers the question we asked earlier: how an **internal weight** changes the **loss** at the end.

GRADIENTS WITH RESPECT TO THE PARAMETERS



For a linear layer $h^\ell = Wh^{\ell-1}$, the local gradients are ordinary calculus:

$$\frac{\partial h^\ell}{\partial h^{\ell-1}} = W \quad \text{and} \quad \frac{\partial h^\ell}{\partial W} = h^{\ell-1, \tau}$$

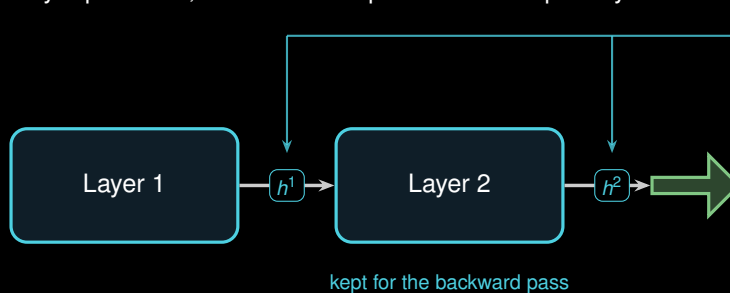
Chain rule: $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$

$$\frac{\partial L}{\partial h^{\ell-1}} = \frac{\partial L}{\partial h^\ell} \cdot \frac{\partial h^\ell}{\partial h^{\ell-1}} \quad \frac{\partial L}{\partial W} = \frac{\partial L}{\partial h^\ell} \cdot \frac{\partial h^\ell}{\partial W}$$

STEP 1: THE FORWARD PASS

Run the mini-batch through the graph, layer by layer, all the way to the loss.

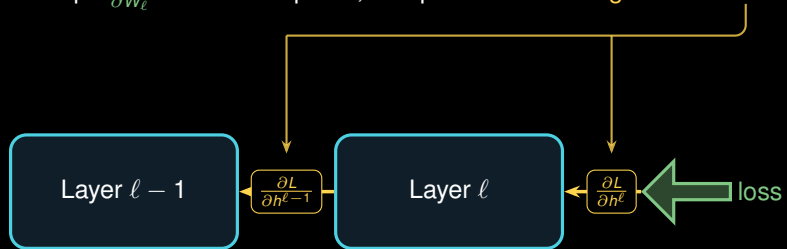
Training uses far more memory than inference does. The gradient equations contain the values each layer produces, so the forward pass has to keep every **intermediate output**.



$$h^\ell = f_\ell(h^{\ell-1}, W_\ell), \quad \ell = 1, \dots, L$$

STEP 2: THE BACKWARD PASS

Start at the **loss**, where the gradient is already known, and walk the layers in reverse. Each layer keeps $\frac{\partial L}{\partial W_\ell}$ for its own update, and passes back the **gradient for its own input**.



STEP 3: UPDATE EVERY PARAMETER

Every gradient is now known, so we can update all the weights.

